Lab 13: AI APIs as Data Tools

INFO 4617/5617 · Week 13 · Chapter 13, AI APIs as Data Tools

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll build one pipeline on the City of Boulder's revenue reports from week 9, the PDFs whose last page is a picture of a table:

- **Part 1, Monday: document understanding**, reading a whole PDF, pictures of tables included. Datalab's API turns it into Markdown a program can read, and we check what it read;
- **Part 2, Wednesday: a model as a function**. Through OpenRouter, a language model reads one page and returns JSON in a shape we define, and we check its numbers against week 9's;
- **Part 3, Friday: search by meaning**. Embeddings, lists of numbers that stand for what a passage is about, find the passages that answer a question. A model answers from those passages and names its sources, or says it can't answer. Last, a look at how MCP and skills describe tools to a model.

No chatbots: every call to a model is one request in and one JSON answer out, and every answer gets checked. Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. You need **two free keys**, each stored as an environment variable by one of week 11's three safe ways (set it, close the terminal, open a new one, and start Jupyter from there):

- `DATALAB_KEY`: sign up at datalab.to, then copy a key from API Keys. The free plan includes a monthly allowance of at least $10 (pricing); this lab spends about 9 cents of it;
- `OPENROUTER_KEY`: sign up at openrouter.ai, then create a key at Keys. The models we use end in `:free`: they cost nothing, up to a daily limit.

Treat each key like a password: never paste it into a cell or a code file. On Wednesday and Friday, start at that day's setup step, or run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Written on 2026-11-07 from the chapter 13 plan (textbook `docs/plans/2026-11-07-ch13-ai-apis.md`, commit 7e083a6 on `claude/ch13-rewrite`) while the chapter itself was being rewritten; its links use the plan's fixed headings, so check them against the chapter once its pull request merges. Run on 2026-11-07 with `labs/tools/run_key.py`: every cell ran, and none is tagged `not-run-live`. Wednesday's and Friday's parts read `saved`, the book's `data/ch-13/` on `main`, which exists only once the chapter's pull request merges: for this run `saved` pointed at the same files on the `claude/ch13-rewrite` branch, and was set back to `main` afterwards without rerunning. A rerun should match Datalab's outputs and costs (4.5 cents a report; without `skip_cache`, a file converted before comes back from Datalab's cache at 0 cents) and the embedding scores, which were the same in two runs. It won't necessarily match the free models: which model answers, and what it gets wrong, change from run to run. Here `nvidia/nemotron-3.5-super-vl:free` answered every request; in Step 10 it matched both totals for 4 of 7 reports, where the plan's check earlier the same day matched 5, with 2018 and 2019 wrong.

Part 1 — Monday: from a PDF to text a program can read

In week 9, pdfplumber read the text layer of Boulder's revenue reports exactly. It stopped at Exhibit 3 of the December 2024 report, a picture of a table, beside a note asking readers to phone the city for "a screen readable version". Today a **document-understanding** API, one that reads a page's

pictures as well as its text, reads the whole report. Then we check what it read, the way week 9 checked pdfplumber.

Step 0 — Set up, and check your keys (5 minutes)

Import what we need, define the User-Agent, and check that both keys are set before reading them, as week 11 did. If one says `MISSING`, quit Jupyter, open a new terminal, and start Jupyter again from there. If you can't restart now, run the given cell: it asks for a missing key with `getpass()`, which doesn't show what you type, and does nothing if both keys are set. The chapter's "Two Keys" has the steps to get them.

```
# Import os, re, time, requests, and pandas (as pd); define HEADERS as in week 12, with your email
# Print whether "DATALAB_KEY" and "OPENROUTER_KEY" are set or MISSING, without printing either key
import os
import re
import time
import requests
import pandas as pd

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("DATALAB_KEY:", "set" if os.environ.get("DATALAB_KEY") else "MISSING")
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
```

```
DATALAB_KEY: set
OPENROUTER_KEY: set
```

```
# Given: only for a key the check said was MISSING, ask for it with getpass. If both said set, this does nothing
from getpass import getpass
for name in ["DATALAB_KEY", "OPENROUTER_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
```

```
# Read DATALAB_KEY into datalab_key; datalab_base = "https://www.datalab.to/api/v1"
# book: the address of week 9's saved PDFs, "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-09/"
datalab_key = os.environ["DATALAB_KEY"]
datalab_base = "https://www.datalab.to/api/v1"
book = "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-09/"
```

💬 Discuss

- Census, FRED and YouTube gave you keys to read their data; Datalab and OpenRouter read whatever you send them. What wouldn't you send?
- Datalab bills by the page, after a free allowance. What does that allowance let the company learn about you?

Step 1 — Submit a document (8 minutes)

Datalab's Convert API takes a PDF, uploaded or by its address, and returns Markdown. We send the address of the book's saved copy of the December 2024 report as `file_url`. We ask for `markdown` in `balanced` mode, and set `paginate` so that each page is marked. Datalab keeps a copy of what it converts, and hands that copy back, free, when the same file comes again. So we also set `skip_cache`: Datalab converts the file afresh, and the cost we see is the real one. The key goes in an `X-API-Key` header. The answer isn't the document. Converting takes seconds, so the API answers at once with

a **ticket**, `request_check_url`: an address where the result will be when it's ready. See the chapter's "Submitting a Document".

```
# data: file_url (book + "revenue-report-2024-12.pdf"), output_format "markdown", mode
 ↪  "balanced", paginate "true", skip_cache "true"
# job: the JSON of a POST to f"{datalab_base}/convert" with data= and headers {"X-API-Key":
 ↪  datalab_key} plus HEADERS; print it
data = {"file_url": book + "revenue-report-2024-12.pdf", "output_format": "markdown",
        "mode": "balanced", "paginate": "true", "skip_cache": "true"}
job = requests.post(f"{datalab_base}/convert", data=data,
                    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(job)
```

```
{'success': True, 'error': None, 'request_id': '8TiORZqhcBr2wJgqyTs-gA', 'request_check_url':
 ↪  'https://www.datalab.to/api/v1/convert/8TiORZqhcBr2wJgqyTs-gA', 'versions': None,
 ↪  'checkpoint_reused': None}
```

💬 Discuss

- Every API so far answered with the data. Why might a service answer with a ticket instead?
- We sent an address, not a file. When would you have to upload the file instead?

Step 2 — Wait for the result (10 minutes)

Ask the ticket's address whether the job is done. The result check answers with a `status`: `processing`, until it's `complete`. Asking again and again until a job is done is called **polling**. Take it in two moves: ask once by hand and print the status; then a given loop asks every two seconds until the job is complete, the way week 12's loop followed the `cursor` until there wasn't one. The finished answer says how many pages it read, `page_count`, and what they cost, `total_cost`, in cents.

```
# check: the JSON of a GET to job["request_check_url"] with the same headers; print its "status"
check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
print(check["status"])
```

```
processing
```

```
# Given: ask again every two seconds until the job is complete; then print what it read and what
 ↪  it cost
while check["status"] != "complete":
    time.sleep(2)
    check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
print(check["page_count"], "pages,", check["total_cost"], "cents,", round(check["runtime"], 1),
 ↪  "seconds of processing")
```

```
15 pages, 4.5 cents, 9.8 seconds of processing
```

💬 Discuss

- The loop has no limit. If Datalab never finished the job, what would your notebook do?
- Fifteen pages cost a few cents. About what would every year-end report Boulder has published cost?
- When would you choose `lite` mode, which cost the chapter about an eighth as much for this report?

Step 3 — Read the Markdown, page by page (12 minutes)

The document is in `check["markdown"]`, one long string. With `paginate`, each page starts with a **page marker**: a number in braces, such as `{14}`, then a line of dashes. In the pattern we split on, `\{\d+\}` matches the number in its braces and `-{10,}` ten or more dashes. So `re.split()` on that pattern gives a list of pages; drop the empty piece before the first marker. Python counts from 0, so page 2 is `pages[1]`. Print page 2, the summary table week 9 read with pdfplumber; then page 15, Exhibit 3, the picture of a table. **Don't print page 1:** the cover gives a staff member's email and phone number, and we keep contact details out of our outputs, as week 9 blacked them out. For the same reason, print Exhibit 3 from its table, which starts at the first `|`, below a note with the sales tax office's phone and email. Its lines are long, so print the first 150 characters of each. See the chapter's "Reading the Markdown".

```
# pages: re.split(r"\n*\{\d+\}-{10,}\n*", markdown) without its first, empty piece; print how
 ↪  many pages
# Print the first 1,200 characters of pages[1], the summary; then of pages[14], Exhibit 3. Never
 ↪  pages[0]
markdown = check["markdown"]
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
print(len(pages), "pages")
print(pages[1][:1200])
```

```
15 pages
This report provides information and analysis related to 2024 unaudited taxes and fees including
 ↪  sales and use tax, accommodations tax, admissions tax, disposable bag fee, electronic smoking
 ↪  device tax, food service tax, marijuana tax, short-term rental tax, sugar sweetened beverage
 ↪  tax, and the trash tax. The revenue reported below was received by the city through January
 ↪  2025 and is largely attributed to economic activity through the month of December. Details by
 ↪  industry, geographic zone and month are found on pages 3 through 7 of this report.

## **Sales and Use Tax Summary**

| <b>Sources - Related to Current Economic Activity</b>               | <b>December YTD</b>  |
 ↪  |                     |                     |                          |
|---------------------------------------------------------------------|----------------------|--┘
 ↪  --------------------|---------------------|---------------------|--------------------------|
|                                                                     | <b>2023</b>          |
 ↪  <b>2024</b>          | <b>\$ Change</b>    | <b>YTD % Change</b> | <b>12 Month % Change</b> |
| Sales Tax
```

```
# exhibit: pages[14] from its first "|" (the table, below the note); print the first 150
 ↪  characters of its first 10 lines
exhibit = pages[14][pages[14].index("|"):]
for line in exhibit.splitlines()[:10]:
    print(line[:150])
```

```
| REVENUE<br>CATEGORY            | YEAR        | JAN               | FEB               | MAR
 ↪  | APR               | MAY               | J
|--------------------------------|-------------|-------------------|-------------------|--------┘
 ↪  -----------|-------------------|-------------------|--
|                                |             |                   |                   |
 ↪  |                   |                   |
| <b>Retail Sales Tax</b>        | 2019        | 8,590,452         | 7,489,624         |
 ↪  9,606,398         | 8,214,047         | 8,832,480         | 1
|                                | 2020        | 7,761,028         | 7,370,943         |
 ↪  10,025,017        | 6,090,136         | 7,059,371         | 9
|                                | 2021        | 8,059,343         | 7,608,759         |
 ↪  10,351,245        | 8,666,637         | 9,229,065         | 1
```

```
|                                | 2022        | 9,592,143         | 9,009,882         |
↪  11,376,046        | 11,242,614        | 10,608,781        | 1
|                                | 2023        | 10,246,130        | 9,156,111         |
↪  12,665,356        | 10,465,614        | 10,686,280        | 1
|                                | <b>2024</b> | <b>10,556,772</b> | <b>9,223,353</b>  |
↪  <b>12,642,682</b> | <b>11,464,301</b> | <b>10,743,033</b> | <
| Change from prior year (month) |             | 3.03%             | 0.73%             | -0.18%
↪  | 9.54%             | 0.53%             | 2
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Datalab doesn't only read text: where a page has a chart, it writes a description of it. Chart 8, on page 9, is a pie chart of marijuana revenue, with its figures printed on the slices.

Do: print page 9 (`pages[8]`), and compare the description under the chart with the figures in its table. **Share:** one sentence of the description that the figures contradict, and who wrote that sentence: the city, or Datalab?

```
# Microsprint: print pages[8]; read the description under Chart 8 against its figures
print(pages[8][:2200])
```

```
### Marijuana Tax Revenues

The City of Boulder collects revenue from the sale of both medical and recreational marijuana.
↪  The city collects the base 3.86% sales and use tax on medical and recreational marijuana. The
↪  city also collects an additional 3.50% sales and use tax on recreational marijuana, and a
↪  5.0% excise tax when marijuana is transferred from a cultivation facility to a dispensary,
↪  testing facility or facility that produces marijuana infused products. In addition, the city
↪  receives a revenue share back from state recreational marijuana taxes, which is reflected in
↪  Chart 8 below.

**Chart 8: Distribution of YTD Marijuana Revenues**

![Pie chart: Breakdown of marijuana revenue by category and
↪  amount.](bedcca5cdf168e3508ef511d94ec514c_img.jpg)

| Category                                                | Amount      |
|---------------------------------------------------------|-------------|
| State Share-Back                                        | \$529,220   |
| Medical Marijuana (3.86% Base)                          | \$52,937    |
| Recreational Marijuana (3.86% Base)                     | \$1,262,688 |
| Recreational Marijuana (3.5% Additional + 5.0% Excise)* | \$1,341,007 |

A pie chart showing the distribution of marijuana revenue. The largest slice is Recreational
↪  Marijuana (3.86% Base) at \$1,262,688, followed by Recreational Marijuana (3.5% Additional +
↪  5.0% Excise)\* at \$1,341,007. State Share-Back is \$529,220, and Medical Marijuana (3.86%
↪  Base) is \$52,937.

Pie chart: Breakdown of marijuana revenue by category and amount.

\*Dedicated to the General Fund.

**Chart 9: Historical Trends for YTD Marijuana Revenues<sup>3</sup>**

![Line chart: Comparison of Medical Marijuana, Rec Marijuana, and Grand Total from 2021 to
↪  2024.](0f3e3ea50bcceb86f6c524ab2b6f3e7a_img.jpg)
```

```
A line chart showing the financial performance of Medical Marijuana, Rec Marijuana, and Grand
 ↪  Total from 2021 to 2024. All three categories show a downward trend over the four-year
 ↪  period.

| Year | Medical Marijuana | Rec Marijuana | Grand Total |
|------|-------------------|---------------|-------------|
| 2021 | \$208,408         | \$5,429,871   | \$5,638,279 |
| 2022 | \$18
```

💬 Discuss

- The city's note asks readers to phone for a screen readable version of Exhibit 3. Who would have to check Datalab's table before the city could publish it as that version?
- The description of Chart 8 calls one slice the largest, then gives a larger one. How would you tell what a model wrote on these pages from what it read off them?
- Page 10's bar chart of sugar-sweetened beverage tax has no figures printed on it. Print `pages[9]`: where could the numbers Datalab wrote beside it have come from?

Step 4 — Check what it read (12 minutes)

Week 9's first check was a sum: a row's months should add up to its total (chapter 9, "Checking an Extraction"). Exhibit 3 has 30 rows of figures, one per tax and year. In the Markdown, each row of the table is one line, with its cells between `|` characters. Split each line on `|`, and strip the `<b>` tags and the commas from its cells. Where `cells[2]` is a year, add the twelve months and compare the sum with the total. The report rounds each figure to the dollar, so allow $5, as week 9 did.

```
# For each line of pages[14]: cells = its pieces split on "|", stripped, without "<b>", "</b>"
 ↪  and ","
# Keep a line whose cells[2] is a year (re.fullmatch(r"20\d\d", ...)); add the 12 months,
 ↪  cells[3:15], and compare with cells[15]
# Count the rows checked, and print any whose months miss the total by more than $5
checked = 0
for line in pages[14].splitlines():
    cells = [c.strip().replace("<b>", "").replace("</b>", "").replace(",", "") for c in
 ↪  line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        months = sum(int(c) for c in cells[3:15])
        checked += 1
        if abs(months - int(cells[15])) > 5:
            print(cells[2], months, cells[15])
print(checked, "rows checked")
```

```
30 rows checked
```

⏱️ Microsprint (5 minutes, alone or in pairs)

The 2018 report is older: its Exhibit 3 is a table of text, not a picture.

Do: submit `revenue-report-2018.pdf` the same way (Steps 1 and 2), and run Step 4's check on its page 15, `pages_2018[14]`. **Share:** its page count and cost, how many rows the check found, and how many missed.

```
# Microsprint: set data["file_url"] to the 2018 report; submit, wait (the loop again), split into
 ↪  pages_2018, and check pages_2018[14]
data["file_url"] = book + "revenue-report-2018.pdf"
job = requests.post(f"{datalab_base}/convert", data=data, headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
```

```
while check["status"] != "complete":
    time.sleep(2)
    check = requests.get(job["request_check_url"], headers={"X-API-Key": datalab_key,
 ↪  **HEADERS}).json()
print(check["page_count"], "pages,", check["total_cost"], "cents")
pages_2018 = re.split(r"\n*\{\d+\}-{10,}\n*", check["markdown"])[1:]
checked = 0
for line in pages_2018[14].splitlines():
    cells = [c.strip().replace("<b>", "").replace("</b>", "").replace(",", "") for c in
 ↪  line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        checked += 1
        if abs(sum(int(c) for c in cells[3:15]) - int(cells[15])) > 5:
            print(cells[2])
print(checked, "rows checked")
```

```
15 pages, 4.5 cents
28 rows checked
```

💬 **Discuss**

- Every row added up. What can a sum not catch? (Week 9's Docling put the right figures beside the wrong label.)
- Week 9 ran Docling on a laptop, free, in about a minute; Datalab took seconds and cost cents. What else differs between the two if the PDF holds a student's records rather than a city's?
- Our code assumes every request worked. If Datalab answered 401 (Unauthorized) because the key was wrong, which line would fail first?

Step 5 — Save the text for Wednesday (3 minutes)

Write the 2024 report's Markdown to a file, so Wednesday doesn't have to pay to read it again. (Wednesday also works from the book's saved copies of all seven year-end reports, made the same way on 2026-11-07, so a student who missed today can start there.)

```
# Write markdown, the 2024 report's, to "revenue-report-2024-12.md" with open(..., "w",
 ↪  encoding="utf-8")
with open("revenue-report-2024-12.md", "w", encoding="utf-8") as f:
    f.write(markdown)
```

End of Part 1

On Wednesday, Part 2 hands one page of this Markdown to a language model and asks for JSON in a shape we define: the summary table's totals. Then we check the model's numbers against week 9's, for seven years of reports. It starts with its own setup step, so you don't need today's cells to have run.

Part 2 — Wednesday: a model as a function

A **language model**, the kind of program behind a chatbot, can be called like any other API: we send it an instruction and one page, and it returns an answer. We won't chat with it. We ask for **structured output**, JSON in a shape we define, so the answer goes straight into a DataFrame. Then we check it, because a model's numbers look right whether or not they are.

Step 6 — Set up again (5 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Import what we need, check and read `OPENROUTER_KEY`, and set two addresses: OpenRouter's API, and the book's saved copies of Datalab's Markdown for the seven year-end reports (`data/ch-13/`), made as on Monday. OpenRouter takes one kind of request and passes it to models from many companies. Models whose ID ends in `:free` cost nothing, but they're shared, so they're sometimes busy.

```
# Import os, re, json, time, requests, and pandas (as pd); define HEADERS; check and read
 ↪  OPENROUTER_KEY
# openrouter_base = "https://openrouter.ai/api/v1"; saved = the book's data/ch-13/ address, as
 ↪  book was Monday's data/ch-09/
import os
import re
import json
import time
import requests
import pandas as pd

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
openrouter_key = os.environ["OPENROUTER_KEY"]
openrouter_base = "https://openrouter.ai/api/v1"
saved = "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-13/"
```

```
OPENROUTER_KEY: set
```

```
# Given: only for a key the check said was MISSING, ask for it with getpass. If both said set,
 ↪  this does nothing
from getpass import getpass
for name in ["OPENROUTER_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
openrouter_key = os.environ["OPENROUTER_KEY"]
```

💬 Discuss

- OpenRouter stands between you and the companies that run the models. What does that make easier?
- With OpenRouter in the middle, who sees each page you send?

Step 7 — Find the summary page (5 minutes)

Read the 2024 report's Markdown from the saved copy, split it into pages as on Monday, and keep the first page that holds the table's last row, `Total Sales and Use Tax`. That one page is all the model will see. It's about 1,500 **tokens**. A token is a word or a piece of one: the unit a model reads and writes text in, and is billed by.

```
# markdown: the .text of a GET to saved + "revenue-report-2024-12.md"; pages: split as on Monday
# summary: the first page that contains "Total Sales and Use Tax"; print its first 1,000
 ↪  characters
markdown = requests.get(saved + "revenue-report-2024-12.md", headers=HEADERS).text
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
summary = [page for page in pages if "Total Sales and Use Tax" in page][0]
print(summary[:1000])
```

```
This report provides information and analysis related to 2024 unaudited taxes and fees including
 ↪  sales and use tax, accommodations tax, admissions tax, disposable bag fee, electronic smoking
 ↪  device tax, food service tax, marijuana tax, short-term rental tax, sugar sweetened beverage
 ↪  tax, and the trash tax. The revenue reported below was received by the city through January
 ↪  2025 and is largely attributed to economic activity through the month of December. Details by
 ↪  industry, geographic zone and month are found on pages 3 through 7 of this report.

## **Sales and Use Tax Summary**

| <b>Sources - Related to Current Economic Activity</b>               | <b>December YTD</b>  |
 ↪  |                     |                     |                         |
|---------------------------------------------------------------------|----------------------|--┘
 ↪  --------------------|--------------------|--------------------|-------------------------|
|
```

💬 **Discuss**

- The phrase is on two pages of every report, and we took the first. What would the model see if that page held a different table?
- What would the model miss if the table ran onto the next page?

Step 8 — One request, one JSON answer (15 minutes)

A request to chat completions holds a list of `messages`: a `system` message with the instruction, then a `user` message with the page. That's the whole conversation; we never add a third message. `response_format` asks for structured outputs. With `type` `json_schema`, the model must answer in the shape of the given `SCHEMA`, a **JSON Schema**: a description, itself in JSON, of the fields an answer must have and the type of each. **Temperature** sets how much chance goes into each word the model picks; `temperature` 0 asks for its most likely answer. We send a **fallback list** of free `models` rather than one model: if the first is busy, OpenRouter tries the next (model fallbacks). The answer's text is in `choices[0]["message"]["content"]`, a JSON string, and `json.loads()` turns it into a dictionary. The response's `usage` counts the tokens the request used. Among them are `reasoning_tokens`: text some models write to work an answer out before they give it, which isn't part of the answer. See the chapter's "One Request, One JSON Object".

```
# Given: the shape of the answer, as a JSON Schema: four fields, each with a type and a
 ↪  description
MONTHS = ["January", "February", "March", "April", "May", "June", "July",
          "August", "September", "October", "November", "December"]
SCHEMA = {
    "type": "object",
    "properties": {
        "month": {"type": "string", "enum": MONTHS,
                  "description": "The month the year-to-date figures run through"},
        "year": {"type": "integer",
                 "description": "The report's current year: the later of the two years the table
                  ↪  compares"},
        "total_prior": {"type": "integer",
                        "description": "The table's last row, Total Sales and Use Tax, for the
                         ↪  earlier year, in dollars"},
        "total_current": {"type": "integer",
                          "description": "The table's last row, Total Sales and Use Tax, for the
                           ↪  report's current year, in dollars"},
    },
    "required": ["month", "year", "total_prior", "total_current"],
    "additionalProperties": False,
}
```

```
INSTRUCTION = ("You extract figures from one page of a City of Boulder revenue report. "
               "Copy numbers exactly as printed, without $ or commas. Use only the page.")
```

```
# MODELS: three free models, in order; body: models, temperature 0, messages (system INSTRUCTION,
 ↪  user summary),
# and response_format {"type": "json_schema", "json_schema": {"name": "totals", "strict": True,
 ↪  "schema": SCHEMA}}
# response: the JSON of a POST to f"{openrouter_base}/chat/completions" with json=body and an
 ↪  Authorization header
# Print the model that answered and its "usage"; answer: json.loads() of the first choice's
 ↪  message content; print it
MODELS = ["nvidia/nemotron-3.5-super-vl:free", "google/gemma-4-31b-it:free",
 ↪  "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
        "messages": [{"role": "system", "content": INSTRUCTION}, {"role": "user", "content":
         ↪  summary}],
        "response_format": {"type": "json_schema", "json_schema": {"name": "totals", "strict":
         ↪  True, "schema": SCHEMA}}}
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response["model"])
print(response["usage"])
answer = json.loads(response["choices"][0]["message"]["content"])
print(answer)
```

```
nvidia/nemotron-3.5-super-vl:free
{'prompt_tokens': 1568, 'completion_tokens': 2466, 'total_tokens': 4034, 'cost': 0, 'is_byok':
 ↪  False, 'prompt_tokens_details': {'cached_tokens': 0, 'cache_write_tokens': 0, 'audio_tokens':
 ↪  0, 'video_tokens': 0}, 'cost_details': {'upstream_inference_cost': 0,
 ↪  'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0},
 ↪  'completion_tokens_details': {'reasoning_tokens': 2413, 'image_tokens': 0, 'audio_tokens':
 ↪  0}}
{'month': 'December', 'year': 2025, 'total_prior': 175517129, 'total_current': 173908431}
```

💬 Discuss

- If a different model answered your neighbor, can you compare your two results?
- Your `usage` shows how many `reasoning_tokens` the model wrote before its answer. Why would they matter on a paid model?
- The free models answer 429, Too Many Requests, when they're busy. If all three did, which line of this cell would fail?

Step 9 — Check it against week 9 (10 minutes)

Week 9 read the same table with pdfplumber, by position, and checked its sums, so we trust its figures. They're our **ground truth**: the answers we check others against. Compare each of the model's fields with them: the two totals, the year, and the month. A model that copies numbers well can still misread which year is which.

```
# Given: week 9's figures for the last row, Total Sales and Use Tax, read with pdfplumber
 ↪  (chapter 9, "The Time Series"):
# for each report, the year before and the report's own year
TRUTH = {
    "revenue-report-2018": (134538892, 140648710), "revenue-report-2019": (140609751, 141954650),
    "revenue-report-2020": (141954650, 135195413), "revenue-report-2021-12": (135195413,
     ↪  152302653),
    "revenue-report-2022-12": (152302653, 169112615), "revenue-report-2023-12": (169112615,
     ↪  175517129),
```

```
    "revenue-report-2024-12": (175517129, 173908431),
}
```

```
# prior, current: TRUTH["revenue-report-2024-12"]; print, for each total, the model's figure,
 ↪  week 9's, and whether they match
# Then print the model's year and month: are they 2024 and December?
prior, current = TRUTH["revenue-report-2024-12"]
print("prior:  ", answer["total_prior"], prior, answer["total_prior"] == prior)
print("current:", answer["total_current"], current, answer["total_current"] == current)
print(answer["year"], answer["month"])
```

```
prior:   175517129 175517129 True
current: 173908431 173908431 True
2025 December
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: send exactly the same request, `body`, three more times, two seconds apart, and print each answer. **Share:** did all three agree, with each other and with week 9? Which field changed, if any?

```
# Microsprint: send body again three times, sleeping 2 seconds between; print the model and the
 ↪  answer each time
for attempt in range(3):
    response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                             headers={"Authorization": f"Bearer {openrouter_key}",
 ↪  **HEADERS}).json()
    print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
    time.sleep(2)
```

```
nvidia/nemotron-3.5-super-vl:free {'month': 'December', 'year': 2025, 'total_prior': 175517129,
 ↪  'total_current': 173908431}

nvidia/nemotron-3.5-super-vl:free {'month': 'December', 'year': 2025, 'total_prior': 175517129,
 ↪  'total_current': 173908431}

nvidia/nemotron-3.5-super-vl:free {'month': 'December', 'year': 2025, 'total_prior': 175517129,
 ↪  'total_current': 173908431}
```

💬 Discuss

- Our three answers agreed with each other, the wrong year included. Does agreement make an answer right?
- `temperature` 0 asks for the most likely answer. Why might your answers still differ from your neighbor's, or from one day to the next?
- If the model had been wrong and you hadn't had week 9's figures, how would you have found out?

Step 10 — Seven reports (15 minutes)

Now the same request for every year-end report from 2018 to 2024. We'll send it seven times, so gather Step 8's lines into a function, `extract_totals(page)`, that takes a page and returns the answer dictionary and the model that gave it. Then loop over `TRUTH`'s reports. For each, read its Markdown, find its summary page as in Step 7, call `extract_totals()`, and compare the answer with week 9's. Pause two seconds between requests. See the chapter's "Seven Reports".

```
# extract_totals(page): Step 8's body and request, with the page as the user message; return the
 ↪  answer and response["model"]
def extract_totals(page):
```

```
    body = {"models": MODELS, "temperature": 0,
            "messages": [{"role": "system", "content": INSTRUCTION}, {"role": "user", "content":
             ↪  page}],
            "response_format": {"type": "json_schema", "json_schema": {"name": "totals",
             ↪  "strict": True, "schema": SCHEMA}}}
    response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                             headers={"Authorization": f"Bearer {openrouter_key}",
↪   **HEADERS}).json()
    return json.loads(response["choices"][0]["message"]["content"]), response["model"]
```

```
# rows: for each report in TRUTH, read its Markdown, find the summary page, extract_totals() it,
 ↪  and add the answer,
# the model, and whether each total matches week 9; sleep 2 seconds between reports. results: a
 ↪  DataFrame; print it
rows = []
for report, (prior, current) in TRUTH.items():
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", requests.get(saved + report + ".md",
↪   headers=HEADERS).text)[1:]
    summary = [page for page in pages if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
    rows.append({"report": report, "model": model, **answer,
                 "prior_ok": answer["total_prior"] == prior, "current_ok":
                  ↪  answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
print(results.drop(columns="model").to_string())
print(results["model"].value_counts())
```

```
                  report     month  year  total_prior  total_current  prior_ok  current_ok
0     revenue-report-2018  December  2018    134538892      139331838      True       False
1     revenue-report-2019   January  2020    141954650      139548151     False       False
2     revenue-report-2020   January  2021    135195413      133478757     False       False
3  revenue-report-2021-12  December  2021    135195413      152302653      True        True
4  revenue-report-2022-12  December  2022    152302653      169112615      True        True
5  revenue-report-2023-12  December  2023    169112615      175517129      True        True
6  revenue-report-2024-12  December  2025    175517129      173908431      True        True
model
nvidia/nemotron-3.5-super-vl:free    7
Name: count, dtype: int64
```

💬 Discuss

- Where on the page did a wrong report's figures come from? (Print its summary page and read the table's last two rows.)
- The 2024 report's year came back as 2025, and a sentence on its page says 2025. Would you fix that in the instruction, the schema's description, or a check afterwards?
- The loop has no error handling. If the fifth request got a 429 from every model in the list, what would `rows` hold when the cell stopped?

End of Part 2

On Friday, Part 3 asks questions of all seven reports at once. Embeddings find the passages that answer a question, and a model answers from them, with the IDs of its sources, or says it can't. Then we look at how MCP and skills describe tools to a model. It starts with its own setup step.

Part 3 — Friday: search by meaning, answer with sources, and tools

Seven reports make about 240 passages. An **embedding** turns a passage into a list of numbers, so that passages about the same thing get similar numbers. A question becomes a list of numbers too, and the passages closest to it are the ones to read. Then a model answers from those passages only, as JSON with the IDs of its sources. Finding the passages is **retrieval**, and writing the answer from them is generation: together, **retrieval-augmented generation**, or **RAG**. Ours keeps the answer to one figure and its sources. Last, a look at how MCP and skills describe tools to a model.

Step 11 — Set up, and cut the reports into chunks (4 minutes)

Part 3 starts fresh: import what we need (now with NumPy too), and read `OPENROUTER_KEY`. An embedding model reads at most a few thousand tokens, and one of these pages, a full table, is longer than that. So the given cell cuts each page into **chunks**: pieces of at most about 2,500 characters, broken only between lines. Each chunk gets an ID made of the report, the page, and the chunk's number on that page, such as `2024-12 p.9 #1`. See the chapter's "Chunks".

```
# Import os, re, json, time, requests, numpy (as np), and pandas (as pd); define HEADERS; check
 ↪  and read OPENROUTER_KEY
# openrouter_base and saved, as on Wednesday
import os
import re
import json
import time
import requests
import numpy as np
import pandas as pd

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("OPENROUTER_KEY:", "set" if os.environ.get("OPENROUTER_KEY") else "MISSING")
openrouter_key = os.environ["OPENROUTER_KEY"]
openrouter_base = "https://openrouter.ai/api/v1"
saved = "https://raw.githubusercontent.com/cuinfoscience/Web-Data-Science-Book/main/data/ch-13/"
```

```
OPENROUTER_KEY: set
```

```
# Given: only for a key the check said was MISSING, ask for it with getpass. If both said set,
 ↪  this does nothing
from getpass import getpass
for name in ["OPENROUTER_KEY"]:
    if not os.environ.get(name):
        os.environ[name] = getpass(f"{name} isn't set. Paste it here: ")
openrouter_key = os.environ["OPENROUTER_KEY"]
```

```
# Given: read the seven reports' Markdown, split each into pages, and each page into chunks of at
 ↪  most
# about 2,500 characters, breaking only between lines; each chunk gets an ID such as "2024-12 p.9
 ↪  #1"
REPORTS = ["revenue-report-2018", "revenue-report-2019", "revenue-report-2020",
 ↪  "revenue-report-2021-12",
           "revenue-report-2022-12", "revenue-report-2023-12", "revenue-report-2024-12"]
chunks = []
for report in REPORTS:
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", requests.get(saved + report + ".md",
 ↪  headers=HEADERS).text)[1:]
    for number, page in enumerate(pages, start=1):
```

```
        text, part = "", 1
        for line in page.splitlines(keepends=True):
            if len(text) + len(line) > 2500 and text:
                chunks.append({"id": f"{report[15:]} p.{number} #{part}", "report": report,
↪  "text": text})
                text, part = "", part + 1
            text += line
        chunks.append({"id": f"{report[15:]} p.{number} #{part}", "report": report, "text":
↪  text})
chunks = pd.DataFrame(chunks)
print(len(chunks), "chunks from", chunks["report"].nunique(), "reports")
```

```
243 chunks from 7 reports
```

Step 12 — Embed, and search by meaning (10 minutes)

OpenRouter's embeddings endpoint takes a `model` and an `input`, a list of texts. It returns one **vector** per text: the list of numbers that is the text's embedding. We use one free model, `EMBEDDER`, for every vector. Two models turn the same text into different numbers, so their vectors can't be compared, and an embedding model never gets a fallback list. Embed one chunk by hand first, and count its numbers. Then the given cell embeds all the chunks, 20 to a request, and scales each vector to length 1. (A vector's length here is the square root of its squared numbers added up, not how many numbers it has.) To score a chunk against a question we use **cosine similarity**, which measures how nearly two vectors point the same way: 1 if they point exactly the same way, less the further apart they point. For vectors of length 1, it's their **dot product**: multiply the two lists number by number, and add up. So `vectors @ query` scores every chunk against a question at once.

```
# EMBEDDER = "nvidia/nemotron-3-embed-1b:free"; one: the JSON of a POST to
↪  f"{openrouter_base}/embeddings"
# with json {"model": EMBEDDER, "input": [chunks["text"][0]]}; print how many numbers its first
↪  embedding holds
EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
one = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
↪  [chunks["text"][0]]},
                    headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(len(one["data"][0]["embedding"]), "numbers")
```

```
2048 numbers
```

```
# Given: embed every chunk, 20 to a request, with a pause between requests; vectors: one row per
↪  chunk
vectors = []
for start in range(0, len(chunks), 20):
    batch = chunks["text"][start:start + 20].tolist()
    response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
↪  batch},
                             headers={"Authorization": f"Bearer {openrouter_key}",
↪  **HEADERS}).json()
    vectors += [item["embedding"] for item in response["data"]]
    time.sleep(1)
vectors = np.array(vectors)
vectors = vectors / np.linalg.norm(vectors, axis=1, keepdims=True)
print(vectors.shape)
```

```
(243, 2048)
```

```
# question: "How much revenue did the state share back from marijuana taxes?"; embed it the same
 ↪  way
# query: its vector, scaled to length 1; scores: vectors @ query; print the five best chunks' IDs
 ↪  and scores
question = "How much revenue did the state share back from marijuana taxes?"
response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
 ↪  [question]},
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
query = np.array(response["data"][0]["embedding"])
query = query / np.linalg.norm(query)
chunks["score"] = vectors @ query
print(chunks.nlargest(5, "score")[["id", "score"]])
```

```
                id     score
41      2019 p.8 #1  0.467471
75      2020 p.9 #1  0.466428
9       2018 p.9 #1  0.463676
187  2023-12 p.9 #1  0.460978
152  2022-12 p.9 #1  0.458975
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: ask two questions of your own the same way: one about a topic (a tax, a reason for a change) and one about a figure (a total in a given year). **Share:** which question found its passage, and which didn't. What do embeddings seem to capture?

```
# Microsprint: embed your own question as above, score the chunks, and print the five best
question = "Total sales and use tax in 2022"
response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input":
 ↪  [question]},
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
query = np.array(response["data"][0]["embedding"])
query = query / np.linalg.norm(query)
chunks["score"] = vectors @ query
print(chunks.nlargest(5, "score")[["id", "score"]])
```

```
                 id     score
142   2022-12 p.2 #1  0.541317
158  2022-12 p.13 #1  0.535483
159  2022-12 p.13 #2  0.510527
143   2022-12 p.2 #2  0.495142
144   2022-12 p.3 #1  0.493902
```

💬 Discuss

- The marijuana question's five best passages came from five different years, none of them 2024. To get 2024's figure, would you reword the question, or keep only 2024's chunks before scoring?
- Why can't you add a second embedding model's vectors to the same table, even for new reports?

Step 13 — Answer with sources, as JSON (12 minutes)

Now the generation half of RAG, kept to one figure. We'll search for more than one question, so first gather Step 12's lines into a function, `search(question)`, that embeds a question and returns every chunk's score. Then keep only the 2024 report's chunks, and take the three best for the question as **passages**, each headed by its ID. Send them with the question and a schema whose `value` may be **null**, JSON's word for "no value" (`None` in Python). A null is the model's way to say the passages

don't print the answer. Then check its answer against the PDF: week 9's pdfplumber finds $529,220 printed on Chart 8. See the chapter's "Retrieval-Augmented Extraction".

```
# Given: the shape of an answer with its sources. value may be null: the model's way to say the passages don't say
ANSWER_SCHEMA = {
    "type": "object",
    "properties": {
        "value": {"type": ["integer", "null"],
                  "description": "The figure, in dollars, exactly as printed in a passage; null if no passage prints it"},
        "source_ids": {"type": "array", "items": {"type": "string"},
                       "description": "The IDs of the passages that print the figure; empty if value is null"},
    },
    "required": ["value", "source_ids"],
    "additionalProperties": False,
}
ANSWER_INSTRUCTION = ("Answer the question using only the passages, each headed by its ID. "
                      "Give a figure only if a passage prints it; an estimate read from a chart doesn't count. "
                      "If no passage prints it, answer null.")
```

```
# search(question): Step 12's lines, gathered: embed the question, scale its vector to length 1, and return vectors @ it
def search(question):
    response = requests.post(f"{openrouter_base}/embeddings", json={"model": EMBEDDER, "input": [question]},
                             headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
    query = np.array(response["data"][0]["embedding"])
    return vectors @ (query / np.linalg.norm(query))
```

```
# question: Step 12's marijuana question; chunks["score"]: search(question)
# top: the three best-scoring chunks of "revenue-report-2024-12"; print their IDs; passages: each "ID: text", joined with blank lines
question = "How much revenue did the state share back from marijuana taxes?"
chunks["score"] = search(question)
top = chunks[chunks["report"] == "revenue-report-2024-12"].nlargest(3, "score")
print(top["id"].tolist())
passages = "\n\n".join(f"{row.id}: {row.text}" for row in top.itertuples())
```

```
['2024-12 p.9 #1', '2024-12 p.9 #2', '2024-12 p.11 #2']
```

```
# body: Wednesday's, with ANSWER_INSTRUCTION, the user message f"Question: {question}\n\n{passages}", and ANSWER_SCHEMA named "answer"
# Send it; print the model that answered and its answer. Is the value 529220, and do its sources include page 9?
MODELS = ["nvidia/nemotron-3.5-super-vl:free", "google/gemma-4-31b-it:free", "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
        "messages": [{"role": "system", "content": ANSWER_INSTRUCTION},
                     {"role": "user", "content": f"Question: {question}\n\n{passages}"}],
        "response_format": {"type": "json_schema", "json_schema": {"name": "answer", "strict": True, "schema": ANSWER_SCHEMA}}}
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
```

```
print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
```

```
nvidia/nemotron-3.5-super-vl:free {'value': 529220, 'source_ids': ['2024-12 p.9 #1']}
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Page 10's chart of sugar-sweetened beverage tax has no figures printed on it. But beside the chart, the saved Markdown we search has a table Datalab wrote of what the bars seem to show, each figure marked with a ~.

Do: ask "How much sugar-sweetened beverage tax did the city collect in December 2024?" the same way: `search()` it, take 2024's three best chunks, and send them. **Share:** did the model answer null, or a figure? If a figure, where did it come from?

```
# Microsprint: set question; rescore the chunks with search(); take 2024's best three; put them
 ↪  in body's user message, and send it
question = "How much sugar-sweetened beverage tax did the city collect in December 2024?"
chunks["score"] = search(question)
top = chunks[chunks["report"] == "revenue-report-2024-12"].nlargest(3, "score")
print(top["id"].tolist())
passages = "\n\n".join(f"{row.id}: {row.text}" for row in top.itertuples())
body["messages"][1]["content"] = f"Question: {question}\n\n{passages}"
response = requests.post(f"{openrouter_base}/chat/completions", json=body,
                         headers={"Authorization": f"Bearer {openrouter_key}", **HEADERS}).json()
print(response["model"], json.loads(response["choices"][0]["message"]["content"]))
```

```
['2024-12 p.2 #1', '2024-12 p.10 #1', '2024-12 p.1 #1']
```

```
nvidia/nemotron-3.5-super-vl:free {'value': None, 'source_ids': ['2024-12 p.10 #1']}
```

💬 Discuss

- If the model gave a figure for the beverage tax, it may have copied one of Datalab's estimates. Which check would have caught it?
- The sources let you check an answer. What would you do with an answer whose sources don't print its value?
- The schema's description says `source_ids` is empty when `value` is null; check whether yours was. What can a schema's types enforce that its descriptions can't?

Step 14 — Tools: MCP and skills (6 minutes)

Some programs let a model call **tools**, such as a search or a database. The **Model Context Protocol** (MCP) is a shared way for a server to say which tools it offers. You ask it with **JSON-RPC**: a request in JSON that names a `method` for the server to run, with an `id` that its answer repeats. The method `tools/list` returns each tool's name, its description, and an `inputSchema`, a JSON Schema like Wednesday's (the specification). DeepWiki runs a public MCP server that needs no key. Send it `initialize`, then `tools/list`. It answers as a short **event stream**: lines of text, each starting with a label such as `event:` or `data:`, so the JSON is on the line that starts with `data:`. A **skill** is plainer still: a folder of instructions in Markdown, whose `name` and `description` tell a model when to read the rest. Read the first lines of a public one, for PDFs, and its headings. See the chapter's "Tools, MCP, and Skills".

```
# mcp: "https://mcp.deepwiki.com/mcp"; mcp_headers: HEADERS plus "Accept": "application/json,
 ↪  text/event-stream"
# hello: {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion":
 ↪  "2025-06-18",
#   "capabilities": {}, "clientInfo": {"name": "info4617", "version": "1.0"}}}; POST it; print
 ↪  the first 300 characters of the .text
```

```
mcp = "https://mcp.deepwiki.com/mcp"
mcp_headers = {**HEADERS, "Accept": "application/json, text/event-stream"}
hello = {"jsonrpc": "2.0", "id": 1, "method": "initialize",
         "params": {"protocolVersion": "2025-06-18", "capabilities": {}, "clientInfo": {"name":
          ↪  "info4617", "version": "1.0"}}}
print(requests.post(mcp, json=hello, headers=mcp_headers).text[:300])
```

```
event: message
data: {"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-06-18","capabilities":{"logging"
 ↪  :{},"prompts":{"listChanged":true},"resources":{"subscribe":false,"listChanged":true},"tools
 ↪  ":{"listChanged":true}},"serverInfo":{"name":"DeepWiki","version":"4.0.2"},"instructions":"D
 ↪  eepW
```

```
# listing: the .text of a POST of {"jsonrpc": "2.0", "id": 2, "method": "tools/list"}; line: its
 ↪  line that starts with "data: "
# For each tool in json.loads() of the line after "data: ", ["result"]["tools"], print its name
 ↪  and its required inputs
listing = requests.post(mcp, json={"jsonrpc": "2.0", "id": 2, "method": "tools/list"},
 ↪  headers=mcp_headers).text
line = [l for l in listing.splitlines() if l.startswith("data: ")][0]
for tool in json.loads(line[6:])["result"]["tools"]:
    print(tool["name"], tool["inputSchema"]["required"])
```

```
ask_wiki_question ['repoName', 'question']
read_wiki_contents ['repoName']
read_wiki_structure ['repoName']
```

```
# skill: the .text of a GET to
 ↪  "https://raw.githubusercontent.com/anthropics/skills/main/skills/pdf/SKILL.md"; print its
 ↪  first 5 lines,
# then the headings that start with "### ", the tools it teaches
skill =
 ↪  requests.get("https://raw.githubusercontent.com/anthropics/skills/main/skills/pdf/SKILL.md",
 ↪  headers=HEADERS).text
print("\n".join(skill.splitlines()[:5]))
print([line for line in skill.splitlines() if line.startswith("### ")])
```

```
---
name: pdf
description: Use this skill whenever the user wants to do anything with PDF files. This includes
 ↪  reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one,
 ↪  splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF
 ↪  forms, encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to make them
 ↪  searchable. If the user mentions a .pdf file or asks to produce one, use this skill.
license: Complete terms in LICENSE.txt
---
['### pypdf - Basic Operations', '### pdfplumber - Text and Table Extraction', '### reportlab -
 ↪  Create PDFs', '### pdftotext (poppler-utils)', '### qpdf', '### pdftk (if available)', '###
 ↪  Extract Text from Scanned PDFs', '### Add Watermark', '### Extract Images', '### Password
 ↪  Protection']
```

💬 Discuss

- An MCP tool's `inputSchema` and Wednesday's `response_format` are both JSON Schema. What does each one constrain: what the model may send, or what it must return?
- The PDF skill tells a model to use pypdf and pdfplumber, week 9's tools. If a model followed it on Boulder's reports, which of week 9's checks would still be yours to run?

Step 15 — What did we learn? (3 minutes)

In a few sentences, as comments:

- Datalab read a picture of a table and wrote descriptions of charts. Which of its output would you publish, and which would you check first?
- On Wednesday, which reports did the model get wrong, and was the problem the model or what we sent it?
- Step 12's marijuana question found pages from five other years before any of 2024's. Judging by your own questions too, what does an embedding seem to capture? What did keeping only one report's chunks add?
- Every call this week returned JSON, and every answer was checked against something. What would you check your own project's model outputs against?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, convert a public PDF you choose, with no private individuals' names: Monday's Steps 1–3.
- Step 2, a schema of your own: Wednesday's `SCHEMA`, with fields your document prints.
- Step 3, extract: Step 8's request, and Step 10's `extract_totals()` for more than one page.
- Step 4, check: Step 4's sums and Step 9's ground truth; find something to check against.
- Step 5, retrieve and answer with sources, including a question your document can't answer: Friday's Steps 11–13.
- Step 6, report the costs: Step 2's `page_count` and `total_cost`, and Step 8's `usage`.

```
# Your answers here, as comments
# 1. Exhibit 3's table: all 30 rows passed week 9's sums (and 2018's 28), so once a person had
 ↪  checked its labels
#    against the image it could be the screen readable version. Never the chart descriptions:
 ↪  Chart 8's calls
#    $1,262,688 the largest slice and then names $1,341,007.
# 2. Both totals matched week 9 for 2021-2024, 4 of 7. For 2018, 2019 and 2020 the model returned
 ↪  the subtotal "based on
#    current economic activity" as the current year's total, and for 2019 and 2020 it put that
 ↪  year's total in the prior
#    column. It took years and months from "received through January ...": 2025 for the 2024
 ↪  report. Each page held the
#    right row, so the problem was the model's reading; only the check against week 9 showed it.
# 3. What a passage is about, and the words it shares with the question: "Total sales and use tax
 ↪  in 2022" found 2022's
#    summary page first, but the marijuana share-back is on every year's page 8 or 9. Keeping
 ↪  2024's chunks picks the year.
# 4. A published table, a hand count of a sample, or a second tool, as week 9 compared pypdf with
 ↪  pdfplumber.
```

Friday: your portfolio checkpoint (15 minutes)

The last 15 minutes go to your project. This week's checkpoint, **Analyze your data**, is due Sunday, November 15:

- **An analysis notebook** that runs top to bottom, with its output visible;
- **results that answer at least part of your question**: the counts or comparisons your proposal named;
- **at least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption;
- **the README's Methods section** and your preliminary **Findings**.

Use the time to run your analysis notebook once, top to bottom, here, where you can ask. If you use a model anywhere in your analysis, say which, when, and what you checked its output against, in

Methods.